Contents

PART 0
PRELUDE 5
1. Philosophy and Approach of the Mosaic Engine 7
2. QUOTES 8
PART 1
PROGRAMMING
FUNDAMENTALS 9
1. INTRODUCING MOSAIC 11
2. VARIABLES AND VALUES 14
3. ERRORS 19
4. FRAMES AND TIME 21
5. CONDITIONALS 25
6. FUNCTIONS PART 1 34
7. LOOPS 37
8. ARRAYS 44
9. ENUMS 48
10. STRUCTS 51
11. FUNCTIONS PART 2 55
PART 2
GAME
FUNDAMENTALS 58
1. RANDOM VALUES 59
2. AXIS ALIGNED BOUNDING BOXES 60
3. RAYCASTS 62
PART 3
MEMORY 65
1. POINTERS 66
2. ALLOCATION 69
3. MEMORY ARENAS 71
PART 4
DATA
STRUCTURES 78

MOSAIC

By Collin Eye

2026 Edition

MOSAIC

PART 0
PRELUDE

1. Philosophy and Approach of the Mosaic Engine

The goal of Mosaic is to provide a simple and low-level environment to start making games. Since the graphics are simple (just colored tiles) and and it makes no assumptions about how your game should be structured, it provides you the opportunity to create and solve problems on your own. The problems may be simple ("how do I see if these tiles collide", "how do I change this color value", "what does it mean to beat a level?") but they are worth learning how to solve and will train you to become a better game designer.

Rather than providing many complex features it leaves the implementation to you as an opportunity to get practice solving problems. A lot of the features an engine gives you are the low-hanging-fruit features, but that means you miss a great opportunity to tackle some approachable problems. And once an engine has solved the easy problems for you there are only hard problems left, and you want some experience solving problems before taking those on.

Mosaic is a game engine that's easy to understand so you can peel back the layers and check underneath the hood. It is pretty close to the bare minimum needed to get a game running on Windows.

2. QUOTES

Making games combines everything that's hard about building a bridge with everything that's hard about writing an opera. Games are operas made out of bridges.

— Frank Lantz

Everything I know about morality and the obligations of man I owe to soccer.

— Albert Camus

Counting is the religion of this generation it is its hope and its salvation.

— Gertrude Stein

You must be satisfied to be very clear and very real and very analytical at any level. You can't take the whole thing.

— Bill Evans

PART 1
PROGRAMMING FUNDAMENTALS

1. INTRODUCING MOSAIC

We'll be writing our programs in the Mosaic game engine. The defining feature of Mosaic is an N by M grid of "tiles", which are squares that can be set to any color.

The first thing to do is make sure that you download the Mosaic engine and unzip it. You'll see a few folders, and the one you want to look for is titled `src.` Inside that is another directory `my_game`. In it you'll see a file called `my_mosaic.cpp`, so go ahead and open that in your text editor (such as Visual Studio).

There are two functions which define a game or program in Mosaic: `MyMosaicInit()` and `MyMosaicUpdate()`.

`MyMosaicInit()` runs one time when the program is launched. This is where you can initialize your data (for example what position does the player start at).

`MyMosaicUpdate()` is a function which runs once per frame.

So think of `MyMosaicUpdate` as a set of instructions that the computer executes repeatedly. This is where we write the majority of our code: we tell it what to draw and how to update the state of our data.

To manipulate the color of the tiles in Mosaic we use the `SetTileColor()` function which takes an x and y coordinate, along with 3 color values between 0 and 1. In computer graphics we represent colors using 4 channels: red, green, blue, and alpha/transparency. Our screen hardware is built out of pixels, which are 3 lights bulbs (red, green, and blue). The way we create colors is by setting the intensity of each of those bulbs. If the intensity is 0 that means the bulb emits no light, and if it's 1 it emits at full intensity.

Let's look at some code. Navigate to the file in the "examples" directory called "mosaic_basic.cpp" You can delete everything in there and replace it with this.

```
void MyMosaicInit() {
}
```

```
void MyMosaicUpdate() {
  SetTileColor(4, 4, 1.0, 0.0, 0.0);
}
```

One of the best ways to learn programming is to experiment and play around with whatever tool you're using. So I recommend you try calling SetTileColor with different parameters. Try setting the color of each corner. Try creating different colors and developing an intuition for how the channels combine (how do you make purple? how do you make orange?).

Here are some other functions we have access to. You can call these inside `MyMosaicInit()` to setup your grid how you like.

This sets the dimensions of the grid by supplying a width and a height. The max width is 255 and the max height is 255

`SetMosaicGridSize(16, 16);`

This specifies if you want to see the grid lines. Often grid lines can be helpful for visualizing where things are, and may be desired if doing something like a tactics or puzzle game. There is an analogous HideGrid() function.

`ShowGrid();`

This specifies the colors of the grid lines. Supply values for the 3 color channels.

`SetGridColor(0.5f, 0.5f, 0.5f);`

This specifies the background color of the screen. Supply values for the 3 color channels. `SetMosaicScreenColor(0.2f, 0.2f, 0.2f);`

Fundamentally all visual software is simply setting the colors for each pixel on the screen. The complexity lies in how you determine which pixels and how to color them. So for example it is simple to take an image and display it: you simply take each of the pixels of the image and set the corresponding color on the monitor. It is more complicated to render a 3D dimensional scene: imagine you have a triangle floating in space. As you move around the scene the perspective on that triangle will change, so you need to find a way to map a position on the triangle in 3D space onto a pixel on the screen. That is beyond

our knowledge right now, but it is the same basic process of setting the colors of pixels.

2. VARIABLES AND VALUES

I said in the beginning that a program is something that transforms data. In the previous section we learned some ways to transform data, namely the data of tiles. This transformation evolved a couple of pieces of data: the x and y coordinates of the tile we wanted to change, and the red, green, and blue color values we wanted to use for that tile. In those examples we supplied the data itself.

Very often we want to store a piece of data and give it a name; then we can use that name to refer to that piece of data. We call that name a "variable".

To create a variable we declare it. Declaring a variable means we tell the compiler "give me some memory to store this data, and anytime I use this name, I'm talking about the data stored there."

The syntax of a variable declaration looks like this:

```
<type> <name>;
```

A variable declaration is a statement, just like our function calls from last time, so it needs to end with a semicolon. It's important to realize that this is a command, it is telling the computer to do some physical work. A variable can have any name that you want, as long as it does not begin with number, and has only alpha numeric characters, except for the underscore.

x, Foo, bigDog, anything_you_want, are all valid variable names. You do want to make sure your variable names are sensible, and tell the user (in this case, you) what data they are storing. But be aware there is no semantic meaning to the variable names. If you call a variable "health" your program doesn't know if you're actually using that as health, a color, or the character's name.

Let's make our first variable then:

```
void MyMosaicUpdate() {
  int32 x;
}
```

That compiles, but it doesn't do anything interesting because we aren't ever using that variable; we aren't even telling it what data to store!

So what is this "int32" business? Well looking at our syntax example we can see it is the type?, but what is a type? A type determines two important things: how much data there is, and how to interpret and use that data. Unlike the name the type cannot be anything you want; it has a strict definition. For now we're using only "built-in" types, but we will learn how to define our own in the chapter on Structs.

So what is data then? Data is a sequence of bits, which represent two possible values: 0 or 1. A bit is smallest possible amount of information we could represent. So to rephrase our definition: data is a sequence of 0s and 1s.

The type of the data then determines how many 0s and 1s there are in the sequence, and how to interpret them.

As you may be able to guess from the name, the type int is short for "integer" (a positive or negative number), and the number tells us how many bits something of this type is (32).

Let's look at how to assign data to a variable:

```
void MyMosaicUpdate() {
  int32 x;
  x = 4;
  SetTileColor(x, 4, 1, 1, 1);
}
```

This is another statement which says "goto the place in memory where x is and copy the value 4 into that location in memory." When we type the name of the variable as an argument to SetTileColor we're saying "goto the memory address and give me the value stored there."

We call this operation *assignment*.

You can combine declaration and assignment into a single line:

```
void MyMosaicUpdate() {
  int32 x = 4;
  SetTileColor(x, 4, 1, 1, 1);
}
```

This is pretty common because we usually know what value we want our data to start at.

Let's now talk about computer *memory*. Memory is a piece of hardware that can store bits by hodling onto an electric charge. The simplest piece of memory would maintain charge if you told it "remember this bit is 1", and would hold no charge if that bit was 0.

Instead of addressing individual bits in memory, we address them in 8-bit chunks; an 8-bit sequence of bits is called a *byte*. That means that our int32 is 4 bytes. So when we create an int32 variable we are telling the computer that we want 4 bytes. When we copy the value 4 into that spot we are telling it to copy over some binary data into 4 bytes.

When making games we're rarely concerned with the binary representation of our data, but it is important to have a conceputal understanding of appreciation for what we are actually telling the computer to do when we create variables. If games are transforming data, then how that data is represented and transformed is important to understand.

Now that we have a variable we can begin to manipulate that data using *expressions*.

An expression is a series of *operations* (such as addition, multiplication) performed on data that produces a single value when it is *evaluated*. When we assign a variable to an expression, we are really assigning the variable that value of that expression when evaluated. That means when we type:

```
int32 x = 2 + 2;
```

First we are getting space in memory for our variable; then we are evaluating 2 + 2; then we are copying the value 4 into our location in memory.

The whole point of having a variable is so we store some data some where that we can transform.

```
void MyMosaicUpdate() {
  int32 x = 4;
  SetTileColor(x, 4, 1, 1, 1);
  x = x + 1;
  SetTileColor(x, 4, 1, 1, 1);
}
```

This may seem strange, and even non-sensical “we’re defining x using x”, but remember that when we use a variable what we’re really using is the value that variable refers to; so when we say:

```
x = x + 1;
```

what we’re doing is: take the value of x and add 1 to it; take the result of that expression and copy in into the place where x is stored. This changes the value that was stored at x to 1 greater than it was.

This is such a common operation we have a shorter way to write it:

```
x += 2; // this is shorthand for x = x + 2;
x++; // x++ ---> x += 1 ---> x = x + 1;
```

You may be wondering “what is modulo?” Modulos gives you the remainder of one number divided by another. So 4 % 2 is 0, 9 % 2 is 1. We’ll appreciate it’s utility soon.

Here’s an example using multiple operators:

```
void MyMosaicUpdate() {
  int32 x = 0;
  SetTileColor(x, 4, 1, 1, 1);
  x += 1; // x is 1
  SetTileColor(x, 4, 1, 1, 1);
  x *= 2; // x is 2
```

```
  SetTileColor(x, 4, 1, 1, 1);
  x = x * x; // x is 4
  SetTileColor(x, 4, 1, 1, 1);
  x--; // x is 3
  SetTileColor(x, 4, 1, 1, 1);
}
```

Of course you can have more than one variable in a program, and you can any variable in an expression.

```
void MyMosaicUpdate() {
  int32 x = 0;
  int32 y = 4;
  SetTileColor(x, y, 1, 1, 1);
  // This isn't saying that x and y refer to the same thing,
  // it's just copying the value of y into x. If y changes
  // that will not change the value of x.
  x = y;
  SetTileColor(x, y, 1, 1, 1);
  y--;
  SetTileColor(x, y, 1, 1, 1);
  y = y + x;
  x = 0;
  SetTileColor(x, y, 1, 1, 1);
}
```

Note that before you can use a variable you must first declare it. For example if you had moved line 3 below line 4 you would get an error.

Here is a list of data types in C++, and I define differently sized variants for them (this is useful because we don't always want to use 32 bits for example if we know our number is never larger than 255).

integer: int64, int32, int16, int8 unsigned integer: uint64, uint32, uint16, uint8 floating point real number: float32/real32, float64/real64

There are more types that that which we'll learn about soon, and we'll be making our own types too, but that should get us started.

3. ERRORS

When compliing your code you may encounter errors that prevent the executable from being generated. The compiler will report error messages to you, and usually you can use the line number to find the file and location of the error. Here are some common errors you may encounter:

- “undeclared identifier” means that you are using a variable name that hasn’t been declared. May also be caused by using a variable type that doesn’t exist. This is commonly caused by a typo.

```
int32 number = 5;
void MyMosaicUpdate() {
  numbre += 1; // wrong name
  x += 1; // undeclared variable
}
```

- “Identifier not found” means you are using a function name that doesn’t exist. For example “Inputpressed” instead of InputPressed.

```
void MyMosaicUpdate() {
   if (InputPresssed(Keyboard, Etc_Space)) {

   }
}
```

- syntax error: missing ‘;’ means you missed a semicolon on the line above where the error is reported.

```
void MyMosaicUpdate() {
   int32 x = 5
   Print("hellow."); // the error message will be on this line saying that it expected
a semicolon before Print.
}
```

- A missing bracket or paretheses can cause a “cascading error” where the complier reports a ton of errors (often in other files). The reason is because

there's a mismatch between the braces or parathentheses, so as it tries to match them it misinterprets which parts of the code belong to which scopes.
- "redefinition; previous definition was" this means you're declaring a variable that has already been declared. For example if you said "int32 number = 5; int32 number = 7;" You've just told the compiler "make 2 things called number and give them different values", but how can it know what "number" means now!

If you feel like you're editing the code file but you aren't seeing your changes, here's some things to look for:

1. make sure you saved your file. (VS will show an asterisk next to the file if it hasn't been saved).
2. Make sure there are no compiler errors. These can happen if you misspell something or forget some punctuation.
3. make sure game.exe is closed before you compile. You can look for game.exe in the task manager if you're having trouble finding it. The compiler will tell you that it failed to write to game.exe if it's already open

4. FRAMES AND TIME

I've said that our function MyMosaicUpdate() is being called from inside a while loop, but so far we haven't really seen any evidence of that. We have been using it as if it runs one time. It has however been running many times, it's just that each time we were telling it to do the same thing. Now we'll use variables to store and change data over time as our update function runs multiple times.

First, instead of declaring my variable inside the scope of MyMosaicUpdate (which means that each time our function runs it we'll declare a variable and give it a value), I'm going to declare it once outside the scope of our function, and then change it's value each time MyMosaicUpdate() executes.

```
real32 realRedWagon = 0.0f;

void MyMosaicUpdate() {
   SetTileColor(4, 4, realRedWagon, 0, 0);
}
```

If you run and compile this code you'll see that nothing happens, and that's because I set realRedWagon to be 0, but never changed it's value. Now I'm going to use an expression so that it changes each time MyMosaicUpdate() is called.

```
void MyMosaicUpdate() {
   realRedWagon += 0.01f;
   SetTileColor(4, 4, realRedWagon, 0, 0);
}
```

Neat! We can see that the color of our tile changes over time.

We call each time it runs our code a "frame". This name comes from film and animation where the illusion of motion is created by displaying still images in a rapid sequence (traditionally 24 frames per second in film). This is exactly the same thing that happens in a game. Monitors, screens, and tvs all update at a certain rate, called the "refresh rate." Our game tells the graphics card

what image to display, and we need to continually update that image to create a sense of smooth motion or change.

Note that if we did something like this:

void MyMosaicUpdate() { real32 realRedWagon = 0; realRedWagon += 0.01f; SetTileColor(4, 4, realRedWagon, 0, 0); }

we wouldn't ever see any change because each time our function is called we create a new variable called realRedWagon, add 0.01 to it, then get rid of it when we finish running thru the scope of our function.

Now there is a very important point to note about this: the rate at which our color changes is dependent on how many frames we display per second (put another way, how many times we update our simulation per second). This is very dependent on the specs of the computer you're running on.

Let's take this code:

```
real32 realRedWagon = 0.0f;

void MyMosaicUpdate() {
   realRedWagon += 0.1f;
   SetTileColor(4, 4, realRedWagon, 0, 0);
}
```

It will always take 10 frames for red to reach a value of 1, but this means that our frame rate will determine how much time it takes to reach that value.

If we're running at 60 fps then red would reach a value of 1 in 1/6th of a second (10 frames / 60 frames per second)

If we're running at 120 fps then red would reach a value of 1 in 1/12th of a second (10 frames / 120 frames per second)

If we're running at 144 fps then red would reach a value of 1 in 1/14th of a second (10 frames / 144 frames per second)

This is actually very undesirable behavior for a real time application. Consider the consequences: our animations now get faster or slower depending on the

device. And we aren't even doing physics simulations which would also vary in speed.

To solve this issue we want to change our value by some increment that will vary depending on our frame rate. For this we use a value called "delta time", which stores the number of seconds between the start of our last frame and the start of our current frame; in other words it stores how many seconds it took our last frame to do everything (get input, update state, render images, play audio, etc).

Let's call the time between the start of one frame and the start of another "deltaTime" (we get this name from physics and calculus where delta is used to represent the change between two values over a period of time).

If we look at our different frame rates we can find the values of deltaTime At 60 FPS deltaTime = 1 / 60 = 16.6ms At 120 FPS deltaTime = 1/120 = 8.3ms At 144 FPS deltaTime = 6.9ms

```
See how our delta time gets smaller the faster our frame rate is. This makes sense
because we're doing more stuff in the same amount of time, so obviously we must be
doing each frame more quickly.
Now if we use this delta time value in our calculation we can achieve "frame rate
independence."
```

In the Mosaic engine we have access to a variable called DeltaTime which is calculated for us.

```
real32 realRedWagon = 0.0f;
void MyMosaicUpdate() {
    SetTileColor(4, 4, red, 0, 0);
    realRedWagon= realRedWagon + DeltaTime;
}
```

We now see that no matter how many FPS our game is running at, realRed-Wagon will reach a value of 1 after 1 second.

We can of course scale deltaTime to achieve different speeds.

realRedWagon = realRedWagon + (2 * DeltaTime);

Now it will get to a value of 1 in half a second.

Experiment with this yourself: create variables for each of the color channels and vary them at different rates using deltaTime. Try creating a variable for the x and y position and then change those values over time using deltaTime.

Note that our tiles are always at integer positions (0, 0), (1, 0), (2, 0) and so on; but we can use a real number when calling SetTileColor, it's just that the decimal/fractional part of the number will get lopped off. So an x position of 1.5 is really just 1, as is 1.9999, and 2.00001 is 2, and so on.

If you haven't tried changing the position yet, please do so. If you have then read on.

Notice that our tile now appears at the new position. This is because every frame we're clearing all the tiles to black. We do this because it's simpler and more efficient to just wipe the screen and redraw everything each frame, even if it's state hasn't actually changed, than it is to keep track of which pixels need to be updated and which ones should stay the same. We want to do so often in graphics programming we have a function for it: ClearTiles() which takes 3 color channel values and sets all the tiles to that color. We do this at the beginning of every frame so that we start from a clean slate.

```
real32 x = 0.0f;
void MyMosaicUpdate() {
    ClearTiles(0, 0, 0);

    SetTileColor(x, 4, 1, 0, 0);
    x = x + (2 * DeltaTime);
}
```

5. CONDITIONALS

So far our programs have have been processes that change over time, but we've been missing a key ability: for our program to make decisions. We call a making a decision about whether code runs or not a "conditional". All sorts of programs require the conditionals, but the reason they're crucial to interactive programs is because we want to respond to input: "if this key is pressed do this"

Let's look at another process that happens over time: cooking. There is a linear sequence of instructions to carry out when cooking but there are also decisions that have to be made. When a recipe says "heat the oven to 400 degrees" it means "don't put the cake in the oven until it's 400 degrees inside."

Conditionals produce discrete changes

The most basic type of conditionals is the "if statement." An if statement has two parts, a "condition" and a "body" which is a block of code that will only run if the condition "is satisfied" (meaning "if it is true").

```
if (<boolean expression>) {
   // code that only runs if the expression is true
}
```

So what is a boolean expression? Just like the arithmetic expressions we've seen, a boolean expression is a sequence of operations that reduce to a single value when fully evaluated. The value they reduce to is called a "boolean", which is a type that is either *true* or *false*.

```
if (true) {
   SetTileColor(4, 4, 1, 0, 0);
}
```

This is not much of an expression because it is always going to be true.

```
if (false) {
     SetTileColor(4, 4, 1, 0, 0);
```

```
     SetTileColor(4, 5, 1, 0, 0);
     SetTileColor(4, 6, 1, 0, 0);
}
```

This code is never going to run because the expression is false.

Just like we did for ints and reals we have a data type that can store a boolean value; it's name is fittingly "bool".

```
bool flag = false;
if (flag) {
   SetTileColor(4, 4, 0, 1, 0);
   flag = false;
}
else {
   flag = true;
}
if (flag) {
   SetTileColor(5, 4, 0, 1, 0);
}
```

Modify this code so that flag starts out as false.

Now that isn't particularly interesting or powerful; all we're doing is manually saying what code we want to run.

What if we can check user input to make changes to our code.

```
// Notice that we declare this variable outside the cope of our function so that it
is "global"
// this way it is not going to be redeclared every frame.
float32 r = 0.0f;

void MyMosaicUpdate() {
    // out InputPressed function takes a device and a special value representing
our keys.
    // you can find a complete list of keys in input.h, but for now we'll just
use Space.
    // When the function evaluates it returns a true value if the key is pressed.
    bool pressed = InputPressed(Keyboard, Input_Space);

    if (pressed) {
        r += 0.1f;
    }
```

```
    SetTileColor(0, 0, r, 0, 0);
}
```

We could rewrite our condition like this to avoid creating the boolean variable, since our if statement will execute if whatever expression is inside it is true, that expression can be the returned result from InputPressed

```
if (InputPressed(Keyboard, Input_Space)) {
    r += 0.1f;
}
```

We can use our input to set a boolean value based on which key is pressed.

```
bool flag = false;

void MyMosaicUpdate() {

    if (InputPressed(Keyboard, Input_Space)) {
        flag = false;
    }
    if (InputPressed(Keyboard, Input_Tab)) {
        flag = true;
    }

    if (flag) {
        SetTileColor(0, 0, 0, 1, 0);
    }
}
```

Or we could toggle that value with a single press, like a light switch.

```
bool flag = false;

void MyMosaicUpdate() {

    if (InputPressed(Keyboard, Input_Space)) {
        flag = !flag;
    }

    if (flag) {
        SetTileColor(0, 0, 0, 1, 0);
```

```
    }
}
```

Here we see the “not” operator which negates a boolean value, turning a true into false, and false into true.

Now let’s introduce our “boolean operators” which we can use to create expressions that will evaluate to either true or false based on the parameters of the equation.

The first operator we’ll look at is “greater than”, written <, which evaluates to true if the left hand side is greater than the right hand side. We’ll use our variable “Time” so that we can see how our equation evaluates differently as our program runs.

```
if (Time > 1.0f) {
   SetTileColor(4, 4, 1, 0, 0);
}
```

Our program runs for 1 second before we run the code inside the if statement.

Naturally we also have “greater than or equal”, “less than”, and “less than or equal”.

```
// If we only used less than and greater than then we'd potentiall have one frame where
// neither condition is true and we don't set our tile. This gives us total coverage.

if (Time <= 1.0f) {
   SetTileColor(4, 4, 1, 0, 0);
}
if (Time > 1.0f) {
   SetTileColor(4, 4, 0, 1, 0);
}
```

Because time only increases this boils down to “if time is <= 1, set the color to red, otherwise set the color to green”. We have another way to express this kind of condition in C++ using an “else” statement.

An else statement has a body like an if, but it doesn’t have a boolean expression; it always runs if the condition in the if statement above evaluated to false.

```
if (Time <= 1.0f) {
   SetTileColor(4, 4, 1, 0, 0);
}
else {
   SetTileColor(4, 4, 0, 1, 0);
}
```

An else statement must immediately precede an if statement. It cannot go on it's own because there is no condition above it to base its evaluation on.

There is another type of conditional called the "else if" statement, which does take a boolean expression like the if, but it only checks that boolean expression if the conditional above it failed.

```
if (Time <= 1.0f) {
   SetTileColor(4, 4, 1, 0, 0);
}
else if (Time <= 2.0f){
  SetTileColor(4, 4, 0, 0, 1);
}
else {
   SetTileColor(4, 4, 0, 1, 0);
}
```

The second conditional will only be evaluated if Time <= 1.0f evaluates to false. We see you can put an else after a else if; we sometimes call this the "default" or "fallback" case; meaning we want the code to run if none of our conditions were satisfied.

Like else statements, an else if must be preceded by either an if, or an else if. Unlike else statements they can be stacked indefinitely.

```
if (Time <= 1.0f) {
   SetTileColor(4, 4, 1, 0, 0);
}
else if (Time <= 2.0f){
   SetTileColor(4, 4, 0, 0, 1);
}
else if (Time <= 3.0f){
   SetTileColor(4, 4, 1, 0, 1);
}
else if (Time <= 4.0f){
```

```
   SetTileColor(4, 4, 0, 1, 1);
}
else {
   SetTileColor(4, 4, 1, 0, 0);
}
```

That's interesting, but note that eventually the program will reach an end state where you have satisfied the last condition and there is nothing left to do. But what if we wanted to create a looping where after we reach the end we start at the beginning?

Instead of relying on the global time we're going to use conditions to reset a timer when we have reached a certain limit. What we'll do is add the deltaTime to our timer every frame, and then when it becomes >= 4 we'll reset the timer.

```
real32 timer = 0;
void MyMosaicUpdate() {
  if (timer <= 1.0f) {
     SetTileColor(4, 4, 1, 0, 0);
  }
  else if (timer <= 2.0f){
     SetTileColor(4, 4, 0, 0, 1);
  }
  else if (timer <= 3.0f){
     SetTileColor(4, 4, 1, 0, 1);
  }
  else if (timer <= 4.0f){
     SetTileColor(4, 4, 0, 1, 1);
  }
  else {
     timer = 0;
  }
  timer += DeltaTime;
}
```

We can also use InputHeld to determine when a key is being held down.

```
real32 red = 0.0f;

void MyMosaicUpdate() {
   if (InputHeld(Keyboard, Input_Space)) {
      red += 0.1f * DeltaTime;
   }
```

```
   SetTileColor(0, 0, red, 0, 0, 0);
}
```

A very powerful tool you have to inspect the behavior of your program is displaying values in real time. There are two ways we can do this: we can print them into the output buffer on Visual Studio, or we can display text on the screen.

To print to the console we'll use the Print command. This function takes a string argument and ouputs it in VS. For example:

Print("Hello, my name is");

We'll learn more about what a string is in the future, but for now you can just think of it as "a list of symbols I want to display."

If we want to print a variable we'll give this string a "format argument", which tells it how we want to display the data. We'll place the format in the string, and after the string we'll give it the data we want to format into the string.

```
int32 n = 5;

Print("The value of n is %d", n);
```

To display it as a floating point value:

```
real32 x = 3.14f;

Print("The value of x is %f", x);

Print("This displays it with less decimal places %.2f", x);
```

We can format multiple times within one string.

```
Print("The value of x is %f and n is %d", x, n);
```

We can use DrawTextTile to start drawing a line of text starting at the position of one of our tiles. You give it a position, a size, a color, and the text to print.

The position will always clamp to the bottom left corner of whatever tile the position is inside.

```
  DrawTextTile(V2(0, 0), 0.3f, RGB(1, 1, 1), "Tile text");
  DrawTextTile(V2(0, 1), 0.3f, RGB(1, 1, 1), "Tile text");
  DrawTextTile(V2(1, 2), 0.3f, RGB(1, 1, 1), "Tile text");
```

We can draw text at a tile position “off” the screen,

DrawTextTile(V2(16, 4), 0.3f, RGB(1, 1, 1), “Tile text”); DrawTextTile(V2(17, 5), 0.3f, RGB(1, 1, 1), “Tile text”);

If you want to print the value of a variable you can use some of the formatting operators. You can look up all of these by searching for string formatting in C.

Some examples:

```
int32 x = 5;

float32 n = 3.1459f;

DrawTextTile(V2(16, 4), 0.3f, RGB(1, 1, 1), "A number %d", x);

DrawTextTile(V2(16, 4), 0.3f, RGB(1, 1, 1), "A real number %f", n);
```

We can also draw text at an arbitrary place on the screen in in normalized space where screen coordinates go from 0 to 1

```
DrawTextScreen(V2(0.5f, 0.5f), 0.02f, RGB(1, 0, 0), "Screen text");
```

6. FUNCTIONS PART 1

Thruout our programming we have been using functions (SetTileColor, SetMosaicGridSize, sinf, InputPressed…) and we have even been defining our own: MyMosaicInit() and MyMosaicUpdate(). A function is a block of code grouped together under a name. This gives us a couple things: 1) it makes it easy to group code by concept and 2) it allows us to reuse blocks of code without actually repeating it, but by simply typing the function name. When we type the name of a function we "call it." When we call a function what happens is that our program enters the scope of the function and runs the code; when it reaches the end of the function it "returns", meaning that our code picks up on where it left off when the function was called. There is a world outside our MyMosaicUpdate(): some code some where is calling the function that we have been writing. The definition of a function looks like this:

```
<type> <name> (<arguments>) {

}
```

The simplest definition of a function is one that has no return value and takes no arguments. A function that doesn't return any value is given a return type "void."

```
void MyFunction() {
  SetTileColor(4, 4, 1, 0, 0);
  SetTileColor(5, 4, 0, 1, 0);
  SetTileColor(6, 4, 0, 0, 1);
}
```

Note that this just the definition of a function, the code inside will not actually run until we call the function. It may seem like this is different behavior than what we've experienced using MyMosaicUpdate, but that's because our engine is calling that function (from mosaic.cpp if you're curious). Also be aware that like with variables the name has no semantic meaning. We colud call this

function anything we wanted: Foo, DoNotColor3Tiles, CreateFullVideoGame. Just like variables we must declare functions before we are able to use them. Once we have defined this function before our MyMosaicUpdate(), we can call it.

You want to make sure that you define your function before you call it. You must define it at the top-most scope.

```
void MyMosaicUpdate() {
 MyFunction();
}
```

Often times we want to get some data to our functions for it to use. We call this data an argument to the function, and we say that we "pass an argument to the function." We are already familiar with how to use functions that takes arguments: SetTileColor, InputPressed, to name a few. Our arguments are specified by declaring variables, so they look like

```
<type> <name> (<type> <name>) {
}
```

Let's look at an example of passing an argument into a function:

```
void DrawThree(int32 x) {
  SetTileColor(x, 4, 1, 0, 0);
  SetTileColor(x + 1, 4, 0, 1, 0);
  SetTileColor(x + 2, 4, 0, 0, 1);
}

void MyMosaicUpdate() {
 DrawThree(4);
}
```

Your functions will have access to all your "global variables" (those defined outside any scope) as long as the definition comes before the definition of the function.

When we call this function our program will look at the arguments, and it'll create a variable within the scope of the function, and copy the value 4 into the variable x. Now we can use that value to compute which 3 tiles we want to color.

Note that the variable only exists within the scope of our function. Therefore if we had another variable called x, that our function DrawThree is talking about a different variable; we are not copying the value 4 into the x declared within MyMosaicUpdate().

```
void MyMosaicUpdate() {
 int32 x = 100;
 DrawThree(4);
}
```

We can define functions as taking multiple variables.

```
void DrawThree(int32 x, int y) {
  SetTileColor(x, y, 1, 0, 0);
  SetTileColor(x + 1, y, 0, 1, 0);
  SetTileColor(x + 2, y, 0, 0, 1);
}

void MyMosaicUpdate() {
 DrawThree(4, 4);
 DrawThree(4, 5);
 DrawThree(6, 6);
}
```

7. LOOPS

I said in the beginning that there are 3 basic elements of computing: data manipulation (variables, expressions), conditional execution, and loops. We have already learned two, and now comes the third.

Loops are fundamental to any interactive program. To make them interacitve we need some way to repeatedly update what is displayed to the user.

Every video game you have ever played has a loop structure something like this:

1. start game
2. get input
3. update game state
4. render images
5. render audio
6. goto step 2.

A loop is a way for us to repeatedly execute a block of code. The simplest type of loop is called a "while loop"; it's called "while" because it means "keep running this block while this is true."

```
while (<boolean expression>) {
  // body of the loop
}
```

When our program is running it will encounter the while, and evaluate the boolean expression. If the expression is false it will simply skip over it; if it's true it will enter the body of the loop and begin executing all the statements inside. When it finishes the last statement it will jump back to the top and evaluate the boolean expression again. If the expression is true it will begin executing the body again, otherwise it will exit. Remember when I said that someone was calling MyMosaicUpdate(). Well they are calling it from within a loop that looks something like this:

```
while (true) {
  GetInput();
  MyMosaicUpdate();
  Render();
}
```

The ability to loop infinitely might be valuable in this case, but we don't want to create a loop that never ends within a frame of our game, because then we'd never end; we would essentially have halted our program forever.

But since we can put a boolean expression inside the while loop we're able to create a block of code that will run a certain number of times.

```
void MyMosaicUpdate() {
  int x = 0;
  while (x < 3) {
    SetTileColor(x, 4, 1, 0, 0);
    x++;
  }
}
```

Let's "unroll" this loop to see exactly what's happening.

iteration |

Iteration	Instructions
0	0 < 3 ?
	SetTileColor
	x = x + 1
1	1 < 3 ?
	SetTileColor
	x = x + 1
2	2 < 3
	SetTileColor
	x = x + 1
3	3 < 3
	Finish

When reading or writing a loop it can be helpful to unroll it like this in your head (or write it out if you want!). With some experience you'll no longer need to do that.

The key insight is: we are running the same block of code, but each time we run it the value of x has changed. This is usually why we want to create loops: repeatedly run some code where at least one parameter of it is changing.

This may seem like a minor victory, "big woop, we can color 4 tiles using 4 lines of code, who cares?" But we can see the payoffs when the number of times we want to iterate increases.

```
void MyMosaicUpdate() {
  int x = 0;
  while (x < 8) {
    SetTileColor(x, 4, 1, 0, 0);
    x++;
  }
}
```

It also becomes easier to change the number of times we want to iterate without having to move or delete our code.

This pattern of creating a variable, looping while some condition is true, and changing the value of that variable every iteration is so common that C++ provides a way to express it more succintly using a for loop.

A for loop is written like this:

```
for (<assignment>; <boolean expression>; <expression>) {
  <statements>
}
```

When our program encounters a for lop it'll execute it like this:

1) Assignment
2) Evaluate bool expression
3) if boolean expression is true goto 4, else goto 7
4) body
5) expression

6) goto 3
7) End

Let's look at how we would rewrite our previous while loop as a for loop

```
void MyMosaicUpdate() {
  for (int x = 0; x < 8; x++) {
    SetTileColor(x, 4, 1, 0, 0);
  }
}
```

While loops are useful when you have some code where the terminating condition is indefinite. A game is a good example of this: we know at some point our game will end, but it's impossible to say ahead of time when that will happen. A for loop is useful when we know how many times we want something loop. Let's now use our changing value of x to calculate the color.

```
void MyMosaicUpdate() {
  for (int x = 0; x < 8; x++) {
    real32 red = x / 8.0f;
    SetTileColor(x, 4, red, 0, 0);
  }
}
```

Just like with any block of code we can put any other block of code inside it.

```
void MyMosaicUpdate() {
  for (int x = 0; x < 8; x++) {
    if (x < 4) {
       SetTileColor(x, 4, 1, 0, 0);
    }
    else {
       SetTileColor(x, 4, 1, 1, 0);
    }
  }
}
```

Any interactive program is an iterative process over time. Within that iteration we have other iterations which are over spaces (physical space, samples of an sound wave, number of entities…) That sentence suggests we can put loops inside of loops, which we can!

```
void MyMosaicUpdate() {
  for (int y = 0; y < 8; y++) {
    for (int x = 0; x < 8; x++) {
       SetTileColor(x, y, 1, 0, 0);
    }
  }
}
```

We'll refer to these as the inner and the outter for loops. For each iteration of the outter for loop, we will do all iterations of the inner for loop. The total number of iterations will be equal to the number of iterations of the outter for loop times the inner for loop (in this case that's 8 * 8 = 64). We then increase the value of y, and repeat the process again. Unrolled it looks looks like this: iteration:

Iteration	Instruction
0	0 < 8
	do inner loop
	y++
1	1 < 8
	do inner loop
	y++
2	2 < 8
	do inner loop
	y++
…	…
7	7 < 8
	do inner loop
	y++
8	8 < 8
	Finish

By running this code we can see that the x and y values will have been set to every tile position on the board. In fact, this is exactly how our ClearTiles() function works. There are more interesting things that we can do tho:

```
void MyMosaicUpdate() {
  for (int y = 0; y < 8; y++) {
    for (int x = 0; x < 8; x++) {
       real32 red = x / 8.0f;
       real32 green = y / 8.0f;
       SetTileColor(x, y, red, green, 0);
    }
  }
}
```

It's not necessary that the variable assigned in our for loop start at 0.

```
void MyMosaicUpdate() {
  for (int y = 4; y < 8; y++) {
    for (int x = 4; x < 8; x++) {
       real32 red = x / 8.0f;
       real32 green = y / 8.0f;
       SetTileColor(x, y, red, green, 0);
    }
  }
}
```

```
void MyMosaicUpdate() {
  for (int y = 2; y < 6; y++) {
    for (int x = 2; x < 6; x++) {
       SetTileColor(x, y, 1, 0, 1);
    }
  }
}
```

Nor is it necessary that they always increment by 1.

```
void MyMosaicUpdate() {
  for (int y = 0; y < 8; y += 2) {
    for (int x = 0; x < 8; x += 2) {
       real32 red = x / 8.0f;
       real32 green = y / 8.0f;
       SetTileColor(x, y, red, green, 0);
    }
  }
}
```

Exercises:

For these exercises I recommend drawing out a grid and labeling the x/y coordinates of each tile; this will give you a better idea of what you want to do at certain values as you iterate. Visualization is a powerful technique. Our programs describe a process which happens over a number of iterations, and sometimes its hard when looking at the code to visualize what that does because a) it happens so quickly and b) it is represented in a very dense way.

Shape printing:

- Write a function that colors N tiles in row R, starting at column C.
- Write a function that colors N tiles diagonally, starting at row R and column C, and going in 1 of 4 directions.
- Draw a box where you only set the colors on the outer tiles

8. ARRAYS

In games the problems we are trying to solve usually involve multiple instances of the same type of data. If you think of the games you've played there are almost always multiple characters, objects, animations, sounds, and so on.

Even in our example from Physics we had two objects you were trying to dodge. Adding a third would be doable, but would involve a bit of code duplication. Adding 10 would be a pain.

An array is a way we can store multiple pieces of data in a named collection, and access them by requesting the "nth" element ("give me the first element, give me the fifth element" and so on).

An array declaration looks like this:

```
// <type> <name>[<expression>];
int32 numbers[10];
```

When we declare an array variable we're telling the computer "I want enough memory to store this many elements of this type." In this example that type is int32, so we're asking for 40 bytes of memory.

An array is just like any variable, it has a type and we're storing a value of that type. The type we're storing is a memory address, so the value is where the array was stored in memory.

Memory Address	Data
0…999	
1000-1004	First int and the start of "numbers" array
1004-1008	Second int
1008-1012	Third int
1012-1040	Fourth thru tenth int

When we have an array we say it has “n elements”; we can talk about the “nth element” or “the element at index n”. To interact with elements of the array we use the bracket operator [].

```
int32 numbers[10];
numbers[0] = 2; // first element
numbers[1] = 1001; // second element
```

It may seem strange that we use 0 for the first element, but we can consult our diagram to see why. If the array starts at address 1000, and we know each element in our array is 4 bytes, then we know the first element is at address 1000, the second is 1004, third at 1008, and so on. That gives us an expression: startaddress + (size * n) = addressofelementn.

One of the most useful traits of arrays is that we can iterate over them using loops.

```
vec2 positions[10];

void MyMosaicInit() {
  for (int i = 0; i < 10; i++) {
    positions[i] = V2(RandiRange(0, 8), RandiRange(0, 8));
  }
}
void MyMosaicUpdate() {
  for (int i = 0; i < 10; i++) {
    SetTileColor(positions[i], 0, 1, 1);
  }
}
```

You can declare a variable and use that for the size of your array, but the value of this variable must be “constant.” There are two ways that we do this in C++, one you can think of as the “old style”, and the other the “new style.”

We’ll start with the new style first.

```
// Another way we can define our capacity is by using a variable we modify
// to be "const", which tells the compiler that the value never
// changes, meaning that the compiler can safely know what the size of our array
// is when the program is compiled.
int32 const positionCapacity = 10;
```

```
vec2 positions[positionCapacity];

// positionCapacity = 100;
// This line of code won't compile because we're changing the value of a const
variable.

void MyMosaicInit() {
  for (int i = 0; i < positionCapacity; i++) {
    positions[i] = V2(RandiRange(0, 8), RandiRange(0, 8));
  }
}
void MyMosaicUpdate() {
  for (int i = 0; i < positionCapacity; i++) {
    SetTileColor(positions[i], 0, 1, 1);
  }
}
```

The old style uses the “preprocessor.” This is a pass the compiler runs over our code before compiling it. One thing you can use the preprocssor for is “macros”, which are simple textual substitutions.

```
//#define positionCapacity 10
// This doesnt create a variable positionCapacity, instead it says when we compile
our code
// everywhere it sees the term positionCapacity it will replace with the characters 10,
// then it will compile that code.
// That means a line like this doesn't even make sense:
// positionCapacity = 100
// because that's the same as saynig "10 = 100;"


vec2 positions[positionCapacity];

void MyMosaicInit() {
  for (int i = 0; i < positionCapacity; i++) {
    positions[i] = V2(RandiRange(0, 8), RandiRange(0, 8));
  }
}
void MyMosaicUpdate() {
  for (int i = 0; i < positionCapacity; i++) {
    SetTileColor(positions[i], 0, 1, 1);
  }
}
```

I do use some macros in the codebase, and we'll learn more about good uses for them.

9. ENUMS

Enums are a conveinent way to create a new type of data that represents a set of possible values. For example you might have a player character that can be in some number of states like “idle, running, jumping, and falling”. You could of course use an integer to represent that data, but that can be rather opaque. Instead you could create a type PlayerState which is known to only have the values that you “enumerate.”

```
enum PlayerState {
 PlayerState_Idle,
 PlayerState_Running,
 PlayerState_Jumping,
 PlayerState_Falling,
};

PlayerState state = PlayerState_Idle;

// inside update ....
if (state == PlayerState_Idle) {
 // do this
}
else if (state == PlayerState_Running) {
 //do that
};
```

So rather than having an int which could be any number you have a variable of type PlayerState which you know can only be one of those 4 values, and the code is more clear because instead of writing “state == 0” to mean idle you can just use the value PlayerState_Idle.

You can also assign values to an enum. By default each enum value will be 1 greater than the previous value, but let’s say you wanted to assign values going up by powers of 2 (this can be very useful as I’ll explain later).

```
enum Flag {
  Flag_CanSpeak = 1,
  Flag_CanFight = 2,
```

```
  Flag_CanShop = 4,
  Flag_CanHeal = 8,
};
```

So now we have 4 possible values, each one is a greater power of 2. If we look at those values in binary we get: 0001 0010 0100 1000

A common way you'll see flags written is using the "left-shift" operator <<, which shifts the bits to the left.

<<	Result
0001 << 1	0010
0001 << 2	0100
0001 << 3	1000
0100 << 1	1000

```
enum Flag {
  Flag_CanSpeak = 1 << 0,
  Flag_CanFight = 1 << 1,
  Flag_CanShop = 1 << 2,
  Flag_CanHeal = 1 << 3
};
```

Notice that each of those numbers simply has the 1 at a higher place. This means you can use binary arithmetic to create a value that is a combination of these enum values.

```
Flag flag = (Flag)(Flag_CanSpeak | Flag_CanShop);
```

Notice the Flag in parentheses. This "casts" our result. The reason this is necessary is because when we use the bitwise-or operator we cast our values to integers so we can operate on them. But that means the result is still an int, but we want our result to be a Flag. So what we do is "cast" it. You can always try to cast any type of data to another type, such as casting an int32 to a float32, or a char to a bool. However not all conversions are always possible.

Note that this is using a single `|` instead of the `||` we use for boolean logic. This operator simply takes two values and does a bitwise-or on each element, producing a final output that has a one in every bit that was 1 in either of the inputs.

A	B	Result
0001	0010	0011

By doing a bit-wise or our flag value tells us that we're representing something that can speak and shop, but isn't able to heal or fight.

If we want to check if something has a flag we can use the & operator.

Just like with booleans we also have the & operator which behaves how you would expect

A	B	Result
1111	0010	0010
0010	0001	0000

We can see that the result will be the bits that A and B have in common. So if they don't have any bits in common the result will be 0, otherwise we know that they must share some bits. So if we wanted to check our flag to see if the character was able to speak we could write it like:

```
if ((flag & Flag_CanSpeak) > 0) {

}
```

10. STRUCTS

We've been using several built-in types such as int32 and float32. Now we're going to learn how to create our own types out of these built-in types. The way we do this is with a struct, which is a way to bundle a group of data together in one type.

```
struct <name> {
    <member variables>
};

struct Guy {
   int32 health;
   int32 x;
   int32 y;
};
```

Remember that when we declare a variable we're telling the compiler that we need enough bytes to store data of that type, and that we'll access that data using the name. So when we declare a variable of type Guy we're asking for enough space to store 3 int32s (which is 12 bytes).

```
Guy myGuy;
```

When we say "myGuy" we're getting the value of that data. Now we haven't assigned any values to it yet, right now we've simply declared the variable, but the data stored there is going to be untouched from whatever it was before we declared our variable. We can use the {} initializer to zero out all the memory

```
Guy myGuy = {};
```

Now we know that all 12 bytes there are set to 0.

How do we actually access this data tho. Since Guy is a collection of data it doesn't really mean anything to just have something of type Guy, we want to get at the "member variables." We do that using the dot operator "."

```
myGuy.health = 100;
myGuy.x = 5;
myGuy.y = 2;
```

What this is doing is telling the compiler to fetch the data for myGuy, and then based off of that to fetch the sub data from there using the variable name. This is similar to an array where the array name tells us the start of the data, and we use the [] operator to offset past the beginning. Unlike an array however the data in a struct can be of different sizes, so we can't just use an index, but use the variable name, and since the compiler knows the types of each member variable it can figure out how many bytes past the beginning to go. So health is 0 bytes past the beginning, x is 4, and y is 8.

If we changed our definition to use smaller data for health

```
struct MyGuy {
    int8 health;
    int32 x;
    int32 y;
};
```

Now our total size is only 9 bytes since health is 8 bits instead of 32. So health is still at 0 bytes, but x is at 1 byte, and y is at 5 bytes.

For example we've already seen and been using a struct: vec2

```
struct vec2 {
   float32 x;
   float32 y;
};
```

Whenever we make a thing of type vec2 we're telling the computer we need enough data to store 2 float32s.

```
struct Ball {
    // Member variables or parameters or fields
    vec2 position;
    vec2 velocity;
};
```

When we declare a variable of a type that's a struct, we are requesting enough memory from the computer to store all the member variable of that struct next to each other in memory.

```
Ball ball = {};
// the {} operator initalizes all the data in a struct to be 0.


void MyMosaicInit() {
    SetMosaicGridSize(40, 40);
    SetMosaicScreenColor(0.2f, 0.2f, 0.2f);
    // We can access the different members of a struct by using the dot operator.
    // notice that we can "drill down" using this operator. We say "take the ball,
    // give me the position, now take that position, and give me x."
    ball.position.x = 5;
    ball.position.y = 9;
    ball.velocity.x = 0;
    ball.velocity.y = 5;
}

void MyMosaicUpdate() {
    ClearTiles(0, 0, 0);
    ball.position.y += ball.velocity.y * DeltaTime;
    if (ball.position.y > 39) {
        ball.velocity.y *= -1;
        ball.position.y = 39;
        // clamp our position so it's not invalid
    }
    SetTileColor(ball.position, 0.3f, 0.8f, 0.7f);
}
```

With structs you can start to group your loose collection of variables into types. And since you can nest those types you can start to group your entire game state into a collection. At first this may seem like just more indirection, but as projects grow in complexity it can be very helpful.

```
struct GameMem {
   MyGuy guy;
   Ball balls[16];
   int32 score;
   //... and so on
};

// now you have just one global variable called Game and can access everything from it.
```

```
GameMem Game = {};
}
```

One organization I like is to group together inputs into one struct. That way I can just look up input any place in the code without having to call functions, which you can imagine gets very annoying if you have multiple things to check, like if you're supporting gamepads and keyboards.

```
struct PlayerInput {
   bool jumpPressed;
   vec2 moveDirection;
};
// this can just get stored in GameMem

void UpdateInput() {
   Game.input.jumpPressed = InputPressed(Keyboard, Input_Space) ||
InputPressed(Gamepad, Input_A);
   if (InputHeld(Keyboard, Input_LeftArrow)) {
      Game.input.moveDirection.x = -1;
   }
   if (InputHeld(Keyboard, Input_RightArrow)) {
      Game.input.moveDirection.x = 1;
   }
   if (InputHeld(Keyboard, Input_UpArrow)) {
      Game.input.moveDirection.y = 1;
   }
   if (InputHeld(Keyboard, Input_DownArrow)) {
      Game.input.moveDirection.y = -1;
   }

   float32 gamepadX = InputAnalogue(Gamepad, Input_LeftStickX);
   float32 gamepadY = InputAnalogue(Gamepad, Input_LeftStickY);
   if (gamepadX != 0 || gamepadY != 0) {
      Game.input.moveDirection.x = gamepadX;
      Game.input.moveDirection.y = gamepadY;
   }
}
```

11. FUNCTIONS PART 2

We've seen already how functions can be used to group code together. Remember that when a function is called our program "steps into" it and begins executing the instructions in the body of that function. And remember that a function can take arguments.

```
//   <name> <arguments>
void MyFunc(int32 n) {
   // body
   for (int i = 0; i < n; i++) {
     Print("Hellow");
   }
}
```

In all the functions we've been writing the "return type" has been void. However we can write functions with different return types:

```
int32 Square(int32 n) {
  return n * n;
}
```

When this function is done evaluating it's going to return as a value of type int32. That means we can do anything with the result that we would normally do with a value, such as store it in a variable, print it, or use it an expression.

```
int32 x = Square(5);
int32 y = Square(4) + 1;
Print("The value of 9 squared is %d", Square(9));
```

Returning values allows us to group calculations together which is an incredibly useful tool.

For example let's say you had a bunch of NPCs in a game and you wanted to see how many of them are alive.

```
struct NPC {
  vec2 position;
  int32 health;
};

NPC npcs[100];

// ... some code that sets up the NPCs

int32 HowManyAlive() {
   int32 aliveCount = 0;
   for (int i = 0; i < 100; i++) {
      if (npcs[i].health > 0) {
         aliveCount++;
      }
   }
   // the return keyword will exit the function and return us the value
   return aliveCount;
}
```

You can return any value type from a function:

```
float32 BiggestX() {
   real32 biggestX = -INFINITY;
   for (int i = 0; i < 100; i++) {
      if (npcs[i].position.x > biggestX) {
         biggestX = npcs[i].position.x;
      }
   }

   return biggestGap;
}

bool PlayerToLeftOfPosition(vec2 position) {
   for (int i = 0; i < 100; i++) {
     if (npcs[i].position.x < position.x) {
       return true;
       // notice that we're using the return keyword early, because once we find
       // a position to the left we don't need to keep searching.
     }
   }

   return false;
}
```

Since you can return any type, that means we can return the value of one of our own struct definitions. Here we write a function that returns us the value of an NPC:

```
NPC MakeNPC() {
    NPC guy = {};
    guy.position = V2(RandfRange(0, 10), RandfRange(0, 10));
    guy.health = 100;
}

// Now if you ever want to make an NPC you can just say
npcs[myIndexToSet] = MakeNPC();
```

This will evaluate the function and return us the value for our NPC which we then copy into our array.

Exercises:

1. write a function that loops over an array of ints and returns the largest value.
2. Write a function that loops over an array of ints and returns the number of even numbers in the array.
3. Write a function that returns a point that has the smallest x and y values of all your NPCs.

PART 2
GAME FUNDAMENTALS

1. RANDOM VALUES

To generate a random integer value you can use `RandiRange()` which takes two arguments, a min and max value.

```
int32 r = RandiRange(0, 100);
Print("%d", r);
```

To generate a random floating point value you can use `RandfRange()`

```
real32 r = RandfRange(-10.0, 10.0f);
Print("%f", r);
```

Let's say you want to generate a vec2 pointing in a random direction. If you did this:

```
vec2 v = V2(0, 0);
v.x = RandfRange(-1.0, 1.0);
v.y = RandfRange(-1.0, 1.0);
```

You have a couple problems. First, it's not normalized, but second, it's possible that that the vector has no magnitude if you generate a value of 0 for both.

You can use this function instead:

```
real32 angle = RandfRange(0.0, _2PI);
// _2PI is a constant in Mosaic representing the value of Pi * 2,
// in other words, 360 degrees.

vec2 v = Rotate(angle, V2(1, 0));
// this function gives you back a vector rotated by angle,
// expressed in radians.

// if you prefer there are functions that can convert between
// radians and degrees, RadToDeg(), and DegToRad()
```

2. AXIS ALIGNED BOUNDING BOXES

An axis-aligned bounding box (hereby referred to as an *AABB*) is defined by two points: the point with the minimum coordinates, and the point with the maximum coordinates, here by referred to as *min* and *max*.

To determine if a point lies within a bounding box we need to check if that point's x coordinate lies between the min and max values on the x-axis, and then do the same for the y-axis. This function in Mosaic does just that. I recommend drawing this out on graph paper to visualize what it is doing and the cases that succeed and those that fail.

```
bool TestPointAABB(vec2 p, vec2 min, vec2 max) {
    return (p.x > min.x && p.x < max.x) &&
        (p.y > min.y && p.y < max.y);
}
```

When testing two AABBs for collision there are two pieces of information we want: 1) whether they overlap or not; 2) what the axis of minimum penetration is so we know which direction to move one of the boxes to resolve the collision so that they no longer overlap.

We have a function that does that in Mosaic called `TestAABBAABB`

To call it we need to provide the following arguments: `bool TestAABBAABB(vec2 minA, vec2 maxA, vec2 minB, vec2 maxB, vec2 *dir)`

We'll get to what the weird asterisk before `dir` means in more detail, but for now you can think of it as "writing out data". Normally we've been getting information from a function by having it return a value, but here we want two pieces of information: do they overlap, and how much do they overlap. The way we write out data is to give the function the memory address of the variable we want to write into, which you can see in the example below.

An usage example of this code:

```
vec2 minA = V2(2, 2);
vec2 maxA = V2(6, 6);
vec2 minB = V2(4, 3);
vec2 maxB = V2(7, 5);


void MyMosaicUpdate() {
// we'll treat the position of each box as being the minimum
if (InputPressed(Input, Input_RightArrow)) {
   minB.x += 1;
}

// the & operator gets the memory address of the variable which is what
// we need to pass to the TestAABBAABB function. This lets the function
// write out the result of the overlap into outDir if there was one.
vec2 outDir = {};
if (TestAABBAABB(minA, maxA, minB, maxB, &outDir)) {
  SetBlockColor(minA, 4, 4, V4(1, 0, 0, 1));
  SetBlockColor(minB, 3, 2, V4(0, 1, 0, 1));
}
else {
  SetBlockColor(minA, 4, 4, V4(1, 0, 0, 1));
  SetBlockColor(minB, 3, 2, V4(0, 0, 1, 1));
}

}
```

3. RAYCASTS

```
struct Brick {
    vec2 position;
    vec2 dimensions;

    vec4 color;

    bool destroyed;
    real32 timeDestroyed;
};

const int32 BrickCapacity = 100;
int32 BrickCount = 0;
Brick bricks[BrickCapacity];

int32 Width = 116;
int32 Height = 64;

void SpawnBrick(vec2 position) {
    Brick brick = {};
    brick.position = position;
    brick.dimensions = V2(5, 2);
    brick.color = V4(0.8f, 0.4f, 0.3f, 1.0f);

    if (BrickCount < BrickCapacity) {
        bricks[BrickCount] = brick;
        BrickCount++;
    }
}

void MyMosaicInit() {
    SetMosaicGridSize(Width, Height);

    for (int i = 0; i < 20; i++) {
        vec2 pos = V2(RandiRange(0, 40),
                      RandiRange(0, 40));
        SpawnBrick(pos);
    }
}


void MyMosaicUpdate() {
    vec2 rayOrigin = V2(20 * ((1 + sinf(Time * 0.5f)) * 0.5f), 40);
    vec2 rayDir = V2(0, -1);
```

```
    {
        real32 closestDist = 99999999;
        int32 closestBrickIndex = -1;

        real32 maxDist = 20;

        for (int i = 0; i < BrickCount; i++) {
            vec2 brickMin = bricks[i].position;
            vec2 brickMax = brickMin + bricks[i].dimensions;

            real32 t = 0;
            // &t means "take the memory address of t", so we're turning it from
a value
            // into the a memory address.
            // This is sometimes called "passing t by reference" whic is really
just saying
            // that we are sharing access to one of our variables with RaycastAABB
so that
            // it's able to write into it.
            // Since our functions can't return multiple values this is a way that
we can
            // get our functions to mutate multiple pieces of data. Here RaycastAABB is
            // going to return true if it hit something, and write into the value of t
to tell
            // us _where_ it hit.
            if (RaycastAABB(brickMin, brickMax, rayOrigin, rayDir, &t)) {
                //vec2 pointHit = rayOrigin + rayDir * t;

                if (t > maxDist) {
                    continue;
                }

                real32 dist = t;

                if (dist < closestDist) {
                    closestDist = dist;
                    closestBrickIndex = i;
                }
            }
        }

        if (closestBrickIndex >= 0) {
            if (!bricks[closestBrickIndex].destroyed) {
                bricks[closestBrickIndex].timeDestroyed = Time;
                bricks[closestBrickIndex].destroyed = true;
            }
        }
```

```
    }

    //Print("Brick Count %d, ", BrickCount);

    for (int i = 0; i < BrickCount; i++) {
        vec2 pos = bricks[i].position;

        if (bricks[i].destroyed) {
            //continue;
            bricks[i].color = V4(1, 0, 0, 1);
        }

        for (int y = 0; y < bricks[i].dimensions.y; y++) {
            for (int x = 0; x < bricks[i].dimensions.x; x++) {
                SetTileColor(pos.x + x, pos.y + y, bricks[i].color.r,
bricks[i].color.g, bricks[i].color.b);
            }
        }
    }

    SetTileColor(rayOrigin, V4(0, 1, 1, 1));
}
```

PART 3
MEMORY

1. POINTERS

We know that when we make a variable we are telling the CPU that we need some memory of a certain size to store our data, and that our variable name can be used to give us the contents at that memory address, or we can copy new data into that address by using the assignment operator =.

When we assign one variable to another we're simply copying over the data, we are not actually saying that the two things will remain equal.

```
int32 x = 5;
int32 y = x;
y = 100;
```

x has not changed it's value.

Whenever we pass a variable to a function we know that we're creating a new variable, and copying the data in, so in this case:

```
void Square(int32 n) {
   n = n * n;
}

int32 x = 5;
Square(x);
```

Square does not actually change the value of the variable x. All we've done is create a copy. But since our program is about transforming data, it is very useful if we're able to pass in the memory address of data instead of a copy. This is where pointers come in.

A pointer is a type, just like int32 or char, the difference is that a pointer stores a numerical value representing a memory address in RAM.

A pointer is specified by giving a type, and the * symbol.

int32* ptr;

The type is saying that this is a memory address that is pointing to an int32. All pointers are the same size, after all, we're just storing a memory address and what's stored there doesn't affect what the address is.

The * can go on either side of the variable. These are both equivalent declarations.

```
int32* ptr;
int32 *ptr;
```

So how do we get the memory address of a variable? For that we use the & operator which operates on a variable and evaluates to its memory address.

```
int32 x = 5;
int32* ptr = &x;
```

Now ptr has been assigned the memory address of x. So we have the memory address, but how can we take a pointer and change the content of what's stored there? To do that we "dereference" the pointer by using the * operator.

The type-name of a pointer tells us what type of data is being stored there. There is another pointer type, "void*" which says it's simply a memory address and we can't assume anything about the data that it's pointing to. We'll see how this is useful in the next section.

```
*ptr = 100;
```

Now the value of x is 100, because ptr "refers" to x. The dereference operator is the inverse of the address operator. Instead of saying "give me the memory address" we say "give me the data at the memory address."

For example:

```
Print("address is %d, value is %d", ptr, *ptr);
```

So let's rewrite our Square function so that it actually changes the contents.

```
void Square(int32* n) {
   *n = *n * *n;
}
```

That's a lot of asterisks, I know. All we're doing tho is saying "set the value at this memory address to the value at this memory address times the value at this memory address." We could make it more readable:

```
void Square(int32* n) {
   int32 value = *n;
   *n = value * value;
}
```

This is the true value of pointers: it allows us to take one piece of data and pass it thru a series of functions that transform it. Remember, you can take a pointer of any variable, so you can use this with our basic data types, but also with structs.

Whenever you pass in the contents of an array to a function what you'll be passing in is a pointer. Since a pointer is just a memory address, it doesn't say anything about how much data is stored after it, just what the type is.

```
int32 numbers[100];

int32 ReturnLargest(int32* array, int32 count) {
   int32 largest = array[0];
   for (int i = 0; i < count; i++) {
    if (array[i] > largest) {
      largest = array[i];
    }
   }
   return largest;
}
```

The default way that C/C++ handles strings are as pointers.

char* myName = "Collin";

2. ALLOCATION

Sometimes we refer to a chunk of memory that we've allocated as a buffer. It is essentially just like an array, except that sometimes we might store things of different types there.

void *data = malloc(64);

This allocates 64 bytes. I could fill it with numbers if I wanted. Or maybe I'd put one struct that's 20 bytes, and another that's 44.

If I want to access a particular part of the memory I'll need to do pointer arithmetic.

First I cast the pointer to an unsigned 8 bit integer so I can do arithmetic on it.

```
void *offsetData = ((u8 *)data) + offset_in_bytes;
```

If I know the type of thing in data I could instead cast it to a pointer of that type and index it just like an array.

`((int32 *)data)[index]`

This is the same thing as: `((u8 *)data) + sizeof(int32) * index`

I can use memcpy to copy over a block of memory into my pointer

```
int32 number = 10;
memcpy(data, &number, sizeof(int32));
```

This will copy over 4 bytes, starting at the address of number, into the beginning of data.

I might not want to copy it into the beginning of data, in which case I could do this:

```
void *offset = ((u8 *)data) + offset_in_bytes;
memcpy(offset, &number, sizeof(int32));
```

Or using the casting ideas we've seen (assuming that this is a buffer of int32s)

```
memcpy(((int32 *)data)[index], &number, sizeof(int32));
```

This is the same thing as saying

```
((int32 *)data)[index] = number;
```

However we could do much bigger bulk transfers tho. If we had a buffer like

```
void *dataA = malloc(128);
void *dataB = malloc(128);
```

I could say:

```
memcpy(dataA, dataB, 128);
```

Which would copy 128 bytes from B into A. Or I could say

```
memcpy(dataA, (u8 *)dataB + 64, 64);
```

which would copy the last 64 bytes of B into the first 64 bytes of A.

If you're making a buffer and you know the type, simply cast the pointer returned from malloc;

```
int32 *numbers = (int32 *)malloc(sizeof(int32) * capacity);
```

Now that you know the type of the buffer you can use it just like an array (cause that's what it is essentially).

3. MEMORY ARENAS

We discussed how heap allocation can be expensive. So we want to reduce the number of heap allocations. One way we do this is with custom allocators.

A memory arena is one of the simplest types of allocators. It works by making one allocation, and then doling out memory from that buffer when requested. A memory arena works like a stack: you can push allocations onto the stack, the arena will then increase the stack pointer for the next allocation you make. This means that allocations on a memory arena are very cheap since all that happens is it returns us a pointer from a previously heap allocated region and then increases the offset from that pointer.

Deallocation is as simple as setting the offset of the memory arena to an earlier point. We don't actually deallocate any memory, but now any new allocations we'll overwrite whatever data was stored there.

Because the arena is a stack we are not able to arbitrarily deallocate a piece of memory. We can roll-back the offset to an earlier point in the stack. So for example if you need to temporarily allocate 100 ints, use them in some calculation, and then forget about them you can accomplish this by recording the offset before you allocate them, then return the offset to that value when you're done. But you can't allocate 100 ints, then 1000 vec2s, and then deallocate the ints without also deallocating the vec2s.

One of the things that makes memory arenas so powerful is they allow us to organize our data by lifetimes. In games we have very clear lifetimes for most pieces of data.

For example let's say you want to know the distance between every enemy and the player; that's clearly data that you only need for the current frame. Likewise you may want to make a string and perform some operation to create a file path and open a file; again, you only need that for one frame.

Other data may last the entire length of your game, such as all the sound effects you could play, or the maximum number of particles you can display. Then there are allocations that may only exist for the duration of a level, or while an animation is playing, and so on.

Per frame memory is very easy to handle with an arena: you just clear the arena at the end of every frame, but it means you can allocate things very cheaply if you know they're impermanent; think of it as a scratchpad allocator.

Here is an example of creating and using a memory arena.

```
MemoryArena arena = {};
int32 colorsCount = 100;
vec4 *colors = NULL;
void MyMosaicInit() {
    SetMosaicGridSize(10, 10);

    AllocateMemoryArena(&arena, Megabytes(8));
    colors = PushArray(&arena, vec4, colorsCount);
    for (int i = 0; i < 100; i++) {
        colors[i].r = RandfRange(0.0f, 1.0f);
        colors[i].b = RandfRange(0.0f, 1.0f);
        colors[i].b = RandfRange(0.0f, 1.0f);
        colors[i].a = 1;
    }
}
void MyMosaicUpdate() {
    int32 index = 0;
    if (InputPressed(Input, Input_Space)) {
        for (int i = 0; i < 100; i++) {
            colors[i].r = RandfRange(0.0f, 1.0f);
            colors[i].b = RandfRange(0.0f, 1.0f);
            colors[i].b = RandfRange(0.0f, 1.0f);
            colors[i].a = 1;
        }
    }
    int32 sizeWhenAllocated = arena.size;
    int32 *numbers = PushArray(&arena, int32, 100);
    for (int i = 0; i < 100; i++) {
      numbers[i] = RandiRange(0, 100);
    }
    // move the size back to where it was before we pushed
    // the numbers.
    arena.size = sizeWhenAllocated;

    for (int y = 0; y < 10; y++) {
```

```
        for (int x = 0; x < 10; x++) {
            SetTileColor(x, y, colors[index]);
            index++;
        }
    }

   // you can use PushSize to push a specific sized block. For example:
   int32 numbersCount = 100;
   int *numbers = (int32 *)PushSize(&arena, numbersCount * sizeof(int32)); // pushes
400 bytes

   // You can also use this to allocate a single struct
   vec2 *oneVector = (vec2 *)PushSize(&arena, sizeof(vec2));

   // since we ofte just want to push blocks of memory of the same type we have
PushArray for that
   int32 *numbers2 = PushArray(&arena, int32, 100);
   // notice that this one does the casting for you and only needs the number of
elements.
}
```

One of the most useful data structures in programming is the array, but there are some shortcomings. An array is a fixed size, so it can't expand as we add more data into it. Many times we'd like to use an array like a list where we can add or remove things at arbitrary points. An array is more like a box with a bunch of slots than it is like a list.

This motivates us to introduce a new data structure: the dynamic array (sometimes called a list, a vector, or bucket array). In Mosaic a DynamicArray needs to be allocated using a MemoryArena. Essentially a DynamicArray works by keeping a list of multiple arrays of some size. So it might start off with one chunk of 10 elements. When we try to add an 11th thing it'll see if it has enough room, and then if not it'll allocate a new array from the arena and put our new element there.

A DynamicArray is the first data structure we've seen which uses a "template type" which is a C++ feature that allows us to use one data structure on multiple types. Essentially what the compiler does is look at each data type we're using our data structure on, then creates a version of that data structure for that type according to the template.

Let's look at some code:

```
DynamicArray<int32> myArray = MakeDynamicArray<int32>(&arena, 10);
```

The template type is given between the angle brackets. Notice we need the template type both on the function and on the type.

What this does is create a DynamicArray that stores int32s, and starts off with a default size of 10.

One way that a DynamicArray is different from an array is that semantically there's nothing "in" the array by default. Even tho we have enough room for 10 ints, if we tried saying

```
myArray[0] = 1001;
```

we would technically be accessing outside the bounds of the used portion of array. So how do we get things into the array? With the PushBack function

```
// PushBack(<pointer to array>, <element to insert>);
PushBack(&myArray, 1001);
Print("Count %d", myArray.count);
```

Our count is now 1 and so we can access the first element using the array operator. PushBack will guarantee that the array expands in size if we're trying to add an element beyond our current capacity. This is why you don't want to manipulate the count of the array directly.

```
myArray[0] = 2;
Print("element is %d", myArray[0]);
```

We can also remove elements from the array which will move the elements around in memory as you'd expect.

```
// RemoveAtIndex(<pointer to array>, <index of element to remove>);
RemoveAtIndex(&myArray, 0);
```

Our array now has a count of 0.

You can use the .count member to iterate over the array

```
for (int32 i = 0; i < myArray.count; i++) {
   Print("element[%d]: %d", i, myArray[i]);
}
```

At its core writing a file is just about copying some bytes in our program's memory into a file on disk so that it can be accessed by other programs (like exporting a model), or ourselves in a different session of our program (like loading a save game).

In mosaic we use this function to write a file: `bool OpenFileForWrite(char *path, FileHandle *file, MAllocator *alloc, uint32 size)`

```
FileHandle file = {};
if (OpenFileForWrite("data/test_file.txt", &file, &arena, Megabytes(4))) {

}
```

We have an allocator so that we have some where to put the data before we commit the memory to disk.

It also provides a number of functions to let us write different types of data. If we want to write out binary data you can use the functions

```
void WriteByte(FileHandle *file, u8 byte);
uint64 WriteBytes(FileHandle *file, u8 *bytes, u64 count);
```

These are the backbones of file writing since at the end of the day all our data is binary, but we may use different encodings (for example whether we store an integer as it's 32 bit binary value, or we write out the ASCII encoding that's human readable).

There's a few other helper functions for writing out binary data:

```
void WriteInt32(FileHandle *file, int32 value);
void WriteFloat32(FileHandle *file, float32 value);
```

Writing binary data can be efficient, but its not very readable. For example it would be a pretty bad fit for a config file. You can write an ASCII string to disk using this function:

```
void WriteString(FileHandle *handle, char *str, uint32 len);
void WriteChar(FileHandle *file, char c);
```

All these functions can be found in `file_io.cpp`

Even in a binary file you may want to write a string, for example a binary save file might have the position of the character along with the name they entered. But how do you know how long the string is? You can first write out the length of the string followed by each character.

```
char *name = "Degran";
int32 nameLen = StringLength(name);

WriteInt32(&file, nameLen);
WriteString(&file, name, nameLen);
```

Of course if we're writing non-binary data what we want to do is convert that data into a string format and write that to disk.

You can do this with the `sprintf` function provided by the C standard library.

`int sprintf(char *str, const char *format, ...);`

The `str` argument is a pointer to the array of chars you're writing into. The `format` argument is like the format strings you've used when printing. It tells us what type of conversion we want to apply to the data. The `...` is a special symbol that is telling us this takes a "variadic" number of arguments (just like our `Print` function).

```
// write out an integer as a string
char numStr[8] = {};
// we clear out the array because strings are "null terminated", meaning
// that C expects a 0 value to denote where the string ends.
// This can be quite a pain! This is why I have a custom string implementation
// in my engine.

int32 number = 1001;
sprintf(numStr, "%d", number);

// this will write "1001" instead of the binary representation to the file.
WriteString(&file, numStr, StringLength(numStr));
```

PART 4
DATA STRUCTURES

The code you've been writing in Mosaic has been at the outer-most layer. If we peel that back we can look we can see that the idea of a grid of tiles is a construct built on top of the features of a lower layer. What Mosaic has been doing is drawing a grid of colored "quads", and we've been manipulating those colors by calling `SetTileColor`. That grid has been centered in the screen and the position of these tiles has been fixed.

To step outside of Mosaic you'll go into `game_code.cpp` and change `#define MOSAIC 1` to `#define MOSAIC 0`. This will skip compiling the `my_mosaic.cpp` file and compile `my_game.cpp` instead. This has a familiar structure where we have an update and an init function.

Outside the Mosaic layer we have the ability to simply render a colored rectangle of any size and any position. This is done using the `DrawRect` function.

```
DrawRect(V2(0, 0), V2(1, 1), RGB(1.0f, 0.3f, 0.3f));
```

We have some other nice drawing primitives too.

```
DrawLine(V2(3, 0), V2(4, 0), 0.025f, V4(1));
DrawLine(V2(3, 0), V2(3, 1), 0.025f, V4(1,0,0,1));

// instead of taking a start and end point, it takes a start point
// and a direction extending off of it.
DrawRay(V2(-5, -5), Rotate(Time, V2(1, 0)), 0.025f, V4(0.0f, 0.5f, 0.8f, 1.0f));

DrawCircle(V2(-5, -5), 1.0f, 0.025f, V4(1));
```

We can also load and draw sprites now.

```
Sprite mySprite;

// the path is always relative to the top-level directory of mosaic.
// I would recommend storing all of your assets in data/
void MyGameInit() {
  LoadSprite("data/textures/galaga_ship.png", &mySprite);
}

void MyGameUpdate() {
  // This sets the background color.
  ClearColor(RGB(0.0f, 0.0f, 0.0f));
```

```
  DrawSprite(V2(0), V2(0.5, 0.5), &mySprite);
}
```

You can change the camera parameters as well.

```
Camera *camera = &Game->camera;
camera->position.x = sinf(Time);
camera->size = 1 + ((1 + sinf(Time)) * 0.5f);

// Always make sure you call UpdateCamera so it can recompute the projection matrix
// at the new size and position.
UpdateCamera(&Game->camera);
```

A free list is a data structure that can help us manage a collection of things by keeping track of which things in that collection are available for reuse.

Let's say you have an array of objects in a game. We know that you might not be using all the objects in that array every frame, so we can keep track of how many we're using and simply increment the count when we want a new one like this:

```
const int32 capacity = 100;
int32 count;
Object objects[capacity];

void SpawnObject() {
   objects[count++] = ...;
}
```

But how do we remove things remove things? Well it's not as simple as decrementing the count, because we don't necessarily want to remove the thing at the end of the list. One option might be to give our object a field that says if it's deleted.

```
struct Object {
   bool deleted;
};

void DeleteObject(int32 index) {
    objects[index].deleted = true;
}
```

Now when you're rendering the objects or updating them you'll want to check if it's been deleted. But now the annoying thing happens when you think about adding them again, because we can't just add on to the end of our array anymore, because maybe there's a deleted object between the start and the end. So we have to go searching for that deleted object. This is where free lists come in. We can keep track of which indices in our array are available for reuse, then when we add we don't have to iterate from the start of our list, we can just grab the index at the end of our free list and use that.

```
int32 freelistCount = 0;
int32 freelist[capacity];

void DeleteObject(int32 index) {
    freelist[freelistCount++] = index;
    objects[index].deleted = true;
}

void SpawnObject() {
   int32 newIndex = 0;
   // check free list
   if (freelistCount > 0) {
       newIndex = freelist[freelistCount - 1];
       freelistCount--;
   }
   else {
     newIndex = count;
   }

   objects[newIndex] = ...;
}
```

A hash table (sometimes called a map or dictionary) is a data structure for associating two pieces of data, a `key` and a `value`. You can think of an array as a hash table where the key is simply the index of the value. Often tho we have more sophisticated types of associations. For example let's say you want a list of abilities which you can access simply by looking up the name of the ability. If each ability has a name and you store them all in an array, to find an ability `A` with name `X` you'd have to search over the entire array, comparing the name of each ability to the one you're searching for. This can be expensive. In computer science we'd call this an `O(n)` search, or `linear search` because the time-complexity grows linearly. If you have `10` things you have to search `10` times potentially, if you have `100` things you have to search `100` times, and so on. Compare this to an array which is `constant time` or `O(1)` where you can instantly look up a piece of data when you have the index.

A hash table is a data structure that can give us `constant time` for an arbitrary type of `key`.

There are many many ways to do this, but the gist of it is this: you have two arrays, one of your `keys`, and one of your `values`. To create an association between them we say that a `key` "maps" to a `value` by using a hashing function on it. A hashing function is simply a function that takes an arbitrary piece of data and converts it into a number. That number can then be used to index into our array of `values`. So essentially what's happening is we take a key like a string "AbilityName" and turn it into an index that'll look up into our list of values.

Of course we have to solve the problem that two strings might hash to the same value. A good hashing function can reduce the probability of this happening, but it's still potentially an issue. For example let's look at a bad hashing function:

```
int32 BadHash(char *string, int32 length) {
   int hash = 0;
   for (int i = 0; i < length; i++) {
      hash += string[i];
   }
```

```
   return hash;
}
```

This has an obvious issue: `Hash("dog", 3) == Hash("god", 3)` because all we're doing is taking the value of each character and summing them. A good hashing function will do something more sophisticated. There is a tradeoff between quality of the hash and performance.

One simple change we could make is to use the current value of the hash.

```
int32 BadHash(char *string, int32 length) {
   int hash = 0;
   for (int i = 0; i < length; i++) {
      hash += hash string[i];
   }

   return hash;
}
```

Not great, but it is an improvement.

Again, there are many ways to implement a hash table, but the version in Mosaic uses what's known as a `probe` to handle the case of collisions. First, we need to keep an array of bools to note if a slot is occupied or not. Then when we hash a key, before we place it in that slot we need to look to see if that slot is already occupied because of a hash collision. If it isn't, then we simply put the value in the slot. But if it is we simply "probe" forward one index at a time. You can read `hashtable.h` for more details.

Now for the API. We create a hash table by specifying the type of our keys and the type of our values.

```
HashTable<char *, int32> map = {};
AllocatehashTable(&map, 1000);
// There is also a version that takes an allocator if you don't want to allocate
with malloc.
```

To add an item you can simply say:

```
HashTableInsert(&map, "alice", 1001);
HashTableInsert(&map, "bob", 101);
```

Now to access an element in our hash table we will look up using our key and the function will return true if it found something for that key, and write out the value into the pointer we provide.

```
int32 value;
if (HashTableGet(&map, "alice", &value)) {
   Print("Found the value!");
}
else {
   Print("Oops, couldn't find the value");
}
```

So for example if we wanted to build a database of abilities that a designer could simply access using the names, we can store our abilities in an array, and then we'll have a hash table that maps the ability name to the index in the array. Then to look up an ability we simply go `name -> index -> array`.

```
struct Ability {
   char *name;
   int32 damage;
   float32 duration;
   bool applyStun;
};

DynamicArray<Ability> abilities;
HashTable<char *, int32> abilityMap;
//...

AllocateHashTable(&abiblityMap, 64);

void AddAbilityToMap(Ability ability) {
   int32 index = PushBack(&abilities, ability);
   HashTableInsert(&abilityMap, abilityName, index);
}

Ability *GetAbility(char *name) {
   int32 index;
   if (HashTableGet(&abilityMap, name, &index)) {
     return &abilities[index];
   }
```

```
    else {
      return NULL;
    }
}
```

We often want a way to refer to a piece of data. The most fundamental way we can do that is with a pointer to its memory address. Sometimes tho a pointer is too low-level for the way we want to reference data tho. For example, if the data moved locations in memory, the pointer would no longer be valid. But what's worse, we have no way from the pointer itself to know if it's been moved. So when we move the data we have to have some way to find all the pointers to it and set them to null.

This is where the *handle* comes in. Handle in this context is just an informal term for an abstraction of a reference. In a simple example we could consider the indices into an array as the handle to the data.

```
int32 numbers[100];
int32 handle = 5; // referring to the thing at element 5
int32 *pointer = &numbers[5];
```

Now this is probably not a very useful handle because it doesn't give us more information than a pointer. So what if we wanted a handle to record how many times an element had been set in an array, in addition to the address. We can combine this with the idea of a free list too also know whether an item in the list of numbers is empty or not.

This might not be necessary for a list of numbers, but let's imagine a more interesting example. We have a game with 3 entities: a player, a collectable, and an enemy. The player runs around and collects coins, and the enemy picks a coin and moves towards it. But what happens if that coin gets grabbed? We need to pick a new coin. But just using a pointer wouldn't help us there, because there's no coin at that memory address anymore. So we need refer to a coin that also tells us when our reference is "out of date." or "stale."

```
struct CoinEntry {
   bool freed;
   int32 generation; // how many times this has been set.
};

struct CoinHandle {
  int32 index;
  int32 generation; // we can compare this to the NumberEntry
```

```
};

struct Coin {
   CoinHandle handle;
   // stores a handle to itself so you can always go from the
   // data to a reference. Same way we can always go from a piece of data
   // to a memory address with & operator.
   vec2 position;
};

int32 coinCount = 0;
CoinEntry coinEntries[100];
Coin coins[100];
int32 coinFreelistCount;
int32 coinFreelist[100];
```

The way we refer to an element in the `coins` array is with a handle which points us into the `coinEntries` array. From there we can determine if an element is free or not, and whether we're referring to the correct version. Imagine a case like this:

```
Add a coin A
Enemy X picks A as its target.
Remove A (meaning we add it to the freelist)
X's reference to A is now stale.
```

We want the handle `A` to know that the thing it's referring to is no longer there.

```
CoinHandle SpawnCoin(vec2 position) {
   // imagine passing in some arguments like the position of the coin
   int32 newCoinIndex = coinCount;
   if (coinFreelistCount > 0) {
      newCoinIndex = coinFreelist[coinFreelistCount - 1];
   }
   else {
      // we're adding a new entry so we want to bump our count.
      coinCount++;
   }

   Coin *coin = &coins[newCoinIndex];
   CoinEntry *entry = &coinEntries[newCoinIndex];

   if (entry->generation == 0) {
     entry->generation = 1;
```

```
   }
   // this tells us how many times this slot has been used. If it's the first
   // time the generation will be 1, that way we know anything with a generation
   // of 0 has never been used.

   CoinHandle handle = {};
   handle.index = newCoinIndex;
   handle.generation = entry->generation;

   coin->position = position;
   coin->handle = handle;

   return handle;
}

// this is the same concept as dereferencing a pointer where we take the
// reference and want to get the data. Here we're taking the handle and
// using it to get a pointer to the actual data of the coin so we can
// change it if we want.
Coin *GetCoin(CoinHandle handle) {
   CoinEntry *entry = coinEntries[handle.index];

   // we can compare the generations and if there's a mismatch we know
   // that our handle is stale.
   if (entry->generation != handle.generation) {
      return NULL;
   }

   return coins[handle.index];
}

void RemoveCoin(CoinHandle handle) {
   Coin *coin = GetCoin(handle);
   if (coin == NULL) {
       return;
   }
   // if the handle doesnt give us a valid coin then we dont want to
   // delete it because there's nothing there to delete.

   coinFreelist[coinFreelistCount++] = handle.index;
   CoinEntry *entry = coinEntries[handle.index];
   entry->generation++;
   // the generation goes up by 1 so any handle we have to the coin
   // will now mismatch.
}
```

Now when we update our enemy we just need them to dereference their handle. If they get a valid coin then we know it still exists and can follow it. If it doesn't, then that means it's been deleted and we need to pick a new coin.

Keep in mind that if we wanted to have our coinArray be tightly packed instead of using a free list, then we'd want our `CoinInfo` to also keep track of our index in the `coins` array because as things are deleted our index might change.

There's a more formal structure for this type of pattern where we treat all our entities this way, sometimes called an `EntityManager`. We'll look at that next.

Every now and then we want to express the idea of something representing multiple types. A common example is in a game where we have an object that collides with a subset of all our possible object types.

Let's take an example:

```
enum ColliderType {
    ColliderType_Player,
    ColliderType_Ghost,
    ColliderType_Wall,
    ColliderType_Box,
};
```

Each body in the game is only one of these types, but they can each collide with multiple types. For example the player collides with walls and boxes, but not ghosts. Now we could say that each object has an array of ColliderTypes which we check against, but there's another way.

Let's leverage the fact that our data is binary. If we have an int32 we know there's 32 bits, and let's say that we use each of those bits to represent our different ColliderTypes. We'll see why very soon.

We can write that:

```
enum ColliderType {
    ColliderType_Player = 1 << 0,
    ColliderType_Ghost = 1 << 1,
    ColliderType_Wall = 1 << 2,
    ColliderType_Box = 1 << 3,
};
```

This expression is known as a "left shit", and it's taking our value and shifting the bits left by the value on the right side. If we view these numbers in their binary representation we have:

```
0001
0010
0100
1000
```

This is useful because now we can use the “bitwise or” operator to combine these values. For example if we said:

```
ColliderType collidesWith = (ColliderType)(ColliderType_Wall | ColliderType_Box)
```

We’re gonna get a value:

```
0100 | 1000 = 1100
```

Now notice that we don’t have an explicit definition for something of ColliderType with the value 6, which is why we need to cast it.

Okay, so we’ve successfully combined multiple types into a single value, but how do we actually use this? That’s where the “bitwise and” operator comes in. Let’s take our collidesWith value and see the result of using the and operator with various values

```
// let's & it with the value for ColliderType_Player
1100 & 0001 = 0000
// Now let's try ColliderType_Ghost
1100 & 0010 = 0000
// Then ColliderType_Wall
1100 & 0100 = 0100
// And finall ColliderType_Box
1100 & 1000 = 1000
```

Notice that when our left-hand side didn’t have the same bit from the right-hand side we got a value of 0, and when they did share a bit in the same place the result was non-zero (technically it was just the value of the bit they shared). So now we can take our collidesWith value and compare it to any type to see if we should test for collision or not.